REDES NEURONALES
MLP Y CNN

Introducción

A lo largo de esta práctica se desarrollarán dos modelos de aprendizaje supervisado, el primero basado en el perceptrón multicapa y el segundo en redes neuronales convolucionales.

Una red neuronal es un modelo de computación el cual intenta simular el comportamiento del cerebro humano. Se pretende otorgar al modelo, el conocimiento necesario para poder resolver problemas por sí mismo. Están compuestas por entradas, funciones (propagación, activación y transferencia) y salidas.

En este caso, se pretende crear un modelo de entrenamiento el cual sea capaz de etiquetar una imagen, siempre que esta pueda pertenecer a los grupos con los que haya sido entrenado.

Para poder entrenar estos modelos de aprendizaje es necesario disponer de una base de datos lo suficientemente amplia. Para ello haremos uso de CIFAR10.

CIFAR 10

Se trata de una base de datos utilizada frecuentemente para el estudio y desarrollo de redes neuronales. Consta de 60.000 imágenes en color y de tamaño 32x32 píxeles.
Las imágenes se dividen en diez grupos, de ahí su nombre. Y estos son: aviones, coches, pájaros, gatos, ciervos, perros, ranas, caballos, barcos y camiones. Ref1.

Entorno de trabajo

Para el desarrollo y el código se hará uso del lenguaje Python en Ubuntu. También se usará *Keras*, una biblioteca de código abierto en Python la cual se utiliza para el desarrollo de modelos de aprendizaje automático especializado en redes neuronales.

Como se ha comentado anteriormente se implementará la base de datos de **Cifar10** y adicionalmente la biblioteca de matplot para poder mostrar los resultados gráficamente. Además de las bibliotecas necesarias de *keras*.

Para la ejecución del código se usa la terminal de Ubuntu con Python 3, y se hace uso de la tarjeta gráfica como GPU para acelerar los procesos. (necesario exportar los paquetes en la terminal antes de ejecutar)

Tarea A

El funcionamiento de un MLP se basa en la propagación de errores y en el autoajuste para reducir el error en cada iteración.

Se distribuye en distintas capas, las cuales están interconectadas entre sí mediante las neuronas:

Capa de entrada: Cada neurona de esta capa representa una característica o dimensión de los datos de entrada iniciales. El número de neuronas se determina en función de la dimensionalidad de estos datos de entrada.

Capas ocultas: Puede haber varias capas ocultas, entre la de entrada y la de salida; donde cada neurona recibe entradas de todas las neuronas de la capa anterior y producen salidas a las capas siguientes. Aquí normalmente es donde se ajustan los pesos de las conexiones más importantes entre nodos.

Capa de salida: Formada por las neuronas que producen la salida de la red. En este caso su número será igual a la cantidad de grupos que queramos diferenciar (10 grupos - 10 neuronas).

A parte de la distribución en capas hay otros factores claves que determinan el funcionamiento del modelo; los pesos de cada neurona. Ref2.

Cada nodo tiene un **peso** asociado (w_i) y determina cuanta influencia tiene la característica que analiza esa neurona en concreto. Además cada una tiene un sesgo asociado (b), que le otorga la capacidad de ajustar su umbral de salida.

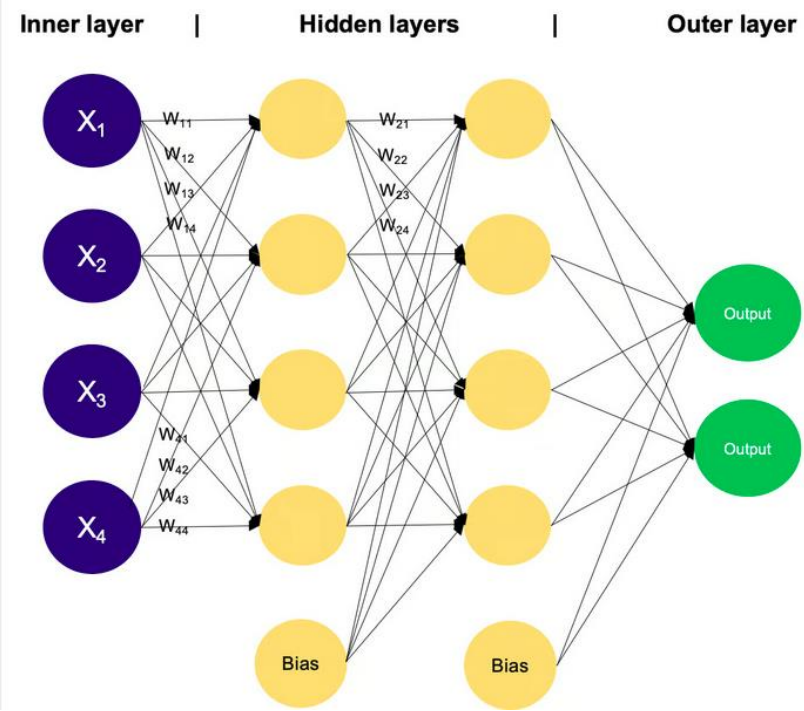
En la capa de salida se calcula la suma ponderada mediante la siguiente fórmula:

$$\textbf{Weighted Sum} = \sum_{i=1}^{n} (w_i * x_i) + b$$

La red debe hacer varios entrenamientos, conocidos como épocas; en los cuales irá ajustando los pesos de las neuronas para reducir el error cometido. Al principio se asignan aleatoriamente los pesos entre las neuronas y tras obtener una salida, se calcula el error cometido; la diferencia entre el resultado obtenido y el esperado.

Luego se hace lo que se conoce como backpropagation, que es el cálculo de cómo cada peso afecta al error mediante el uso de gradientes desde la capa de salida hasta la primera.

Por último, el optimizador escogido, en este caso **sigmoid**, usa los gradientes para ajustar los pesos para poder reducir el error. Esto se repite en cada una de las épocas que se ejecute mejorando la precisión de la red. Ref3.



Para nuestro modelo de MLP debemos declararlo mediante *Keras*. Este primer modelo solo tiene una capa oculta de 32 neuronas, con función de activación “sigmoid” y con 10 neuronas para la capa de salida.

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 3072)	0
dense (Dense)	(None, 32)	98,336
activation (Activation)	(None, 32)	0
dense_1 (Dense)	(None, 10)	330
activation_1 (Activation)	(None, 10)	0

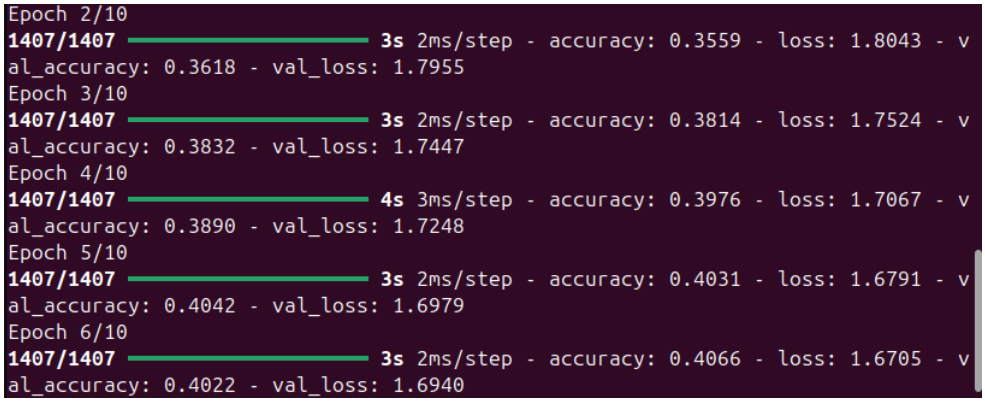
Para poder comenzar el entrenamiento del modelo también hay que cargar la base de datos de imágenes de *Cifar 10*; las cuales se dividen en:

Entrenamiento: 50.000 imágenes a color de 32x32 y las etiquetas de cada imagen.

Test: igual que entrenamiento pero solo con 10.000 imágenes. Son las que se usarán para comprobar la calidad del entrenamiento.

Los datos se deben preprocesar para que el modelo pueda trabajar con ellos. Las imágenes se deben pasar como array y convertir las etiquetas en *one-hot*.

Una vez compilado el modelo y teniendo la base de datos se ejecuta el entrenamiento. Se puede ver el progreso que va habiendo y como va mejorando la tasa de acierto.



Una vez realiza las iteraciones que se han seleccionado previamente, se puede obtener un resumen de la calidad del entrenamiento (*model.fit*) en el que aparece la tasa de acierto y de pérdida con respecto al conjunto de **prueba**.

```
Test accuracy and loss: 0.4235999882221222 1.6279330253601074
```

Para poder visualizar mejor los resultados obtenidos y como ha sido la evolución se han generado las siguientes gráficas, mostrando los siguientes valores:

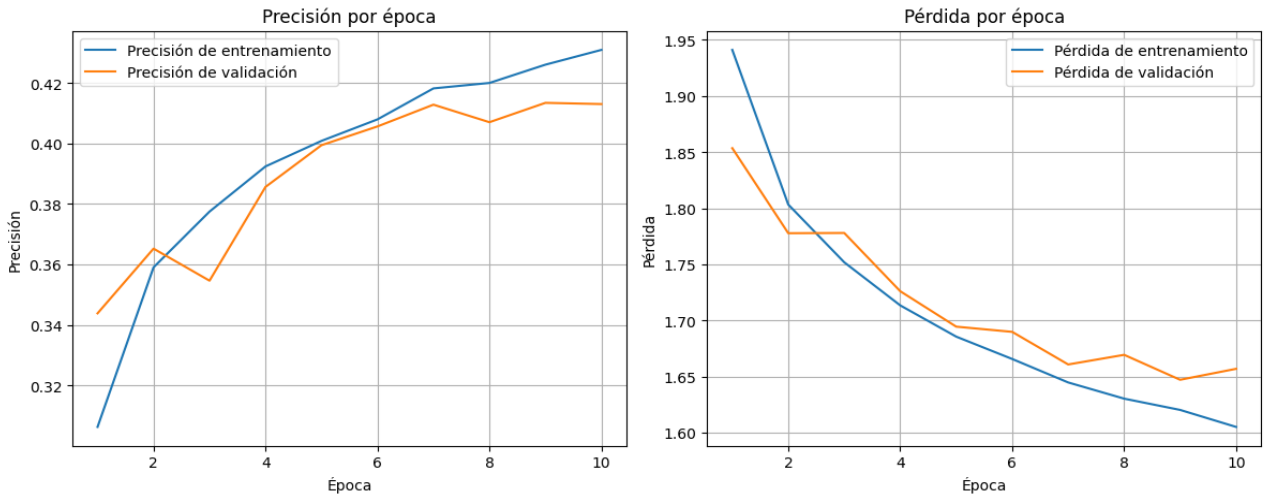


Ilustración 1(train_accuracy, validation_accuracy <>train_loss, validatiomn__loss)

Tarea B

En esta tarea se analizará el comportamiento del MLP creado anteriormente y se estudiará si ha tenido el entrenamiento necesario o por el contrario ha habido sobre entrenamiento.

El sobre entrenamiento es un problema que surge cuando el modelo se ajusta demasiado a las características del conjunto de entrenamiento y pierde generalidad. Es decir, se adapta únicamente a estas características y no es capaz de clasificar bien nuevas imágenes sobre las que no ha entrenado.

Esto se puede determinar viendo el resultado obtenido en una gráfica; si se dejan las suficientes épocas se verá que la curva que representa la tasa de acierto sobre el conjunto de validación deja de aumentar y comienza a descender o se mantiene constante.

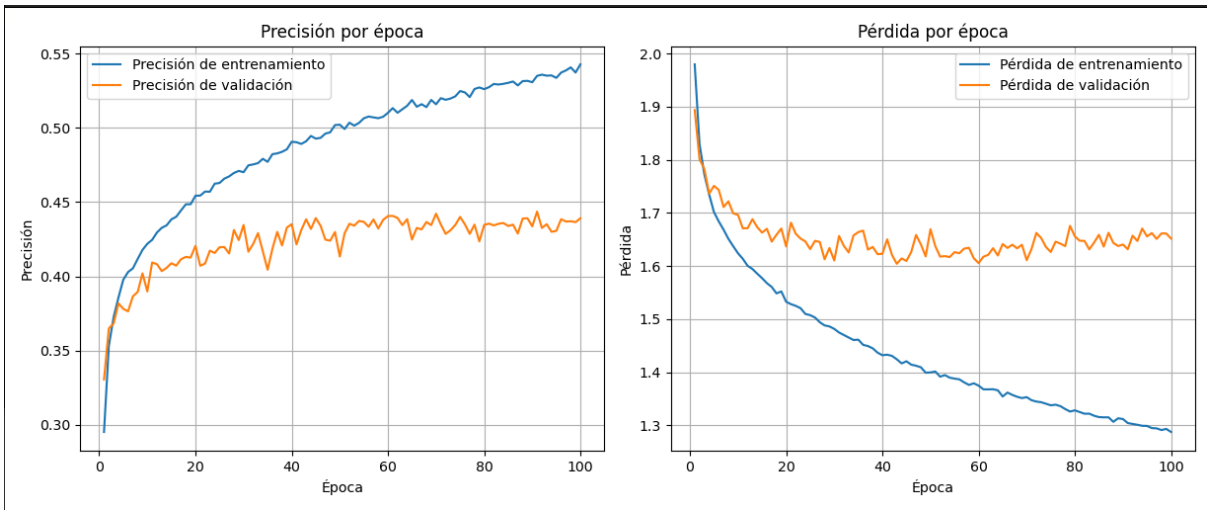


Ilustración 2(Curva precisión/época)

Como se puede observar en la imagen, vemos que la curva de validación se vuelve lineal y no hay mejora. Por tanto, alrededor de 20-25 épocas debería finalizar el entrenamiento para que no se produzca este sobreajuste.

Las épocas hacen referencia a las iteraciones que se ejecutan durante el entrenamiento; es decir, cada vez que se actualizan los pesos de las neuronas y se repite el algoritmo.

Para evitar esto se tiene que concluir el entrenamiento cuando la pendiente tiende a 0, cuando deja de producirse una mejora en la tasa de acierto del conjunto de validación.

Esto se puede solucionar añadiendo un parámetro a la función que se encarga de entrenar el modelo: ***early stop***.

Este parámetro corta la ejecución del entrenamiento cuando detecta que no hay una mejora, evitando así que se produzca sobre entrenamiento y reduciendo el tiempo de ejecución.

La predicción que usa **early stop** se basa en si ha habido o no mejora en las últimas épocas. Hay dos parámetros importantes: paciencia y delta mínimo.

Paciencia: número de épocas que espera sin mejora antes de detenerse.

Delta mínimo: cambio mínimo considerado mejora, osea si es o no una mejora significativa.

Ejecutando el entrenamiento con **early stop** con paciencia 3 y **min_delta** 0.01 nos entrega el siguiente resultado:

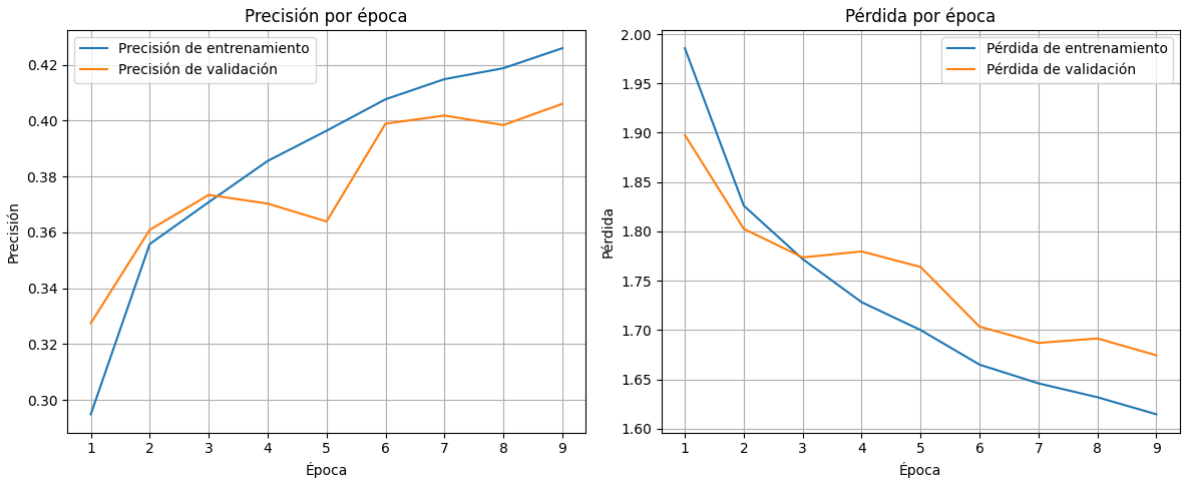


Ilustración 3(Early Stop sin ajuste)

Como se puede apreciar en la ilustración 3, el entrenamiento termina prematuramente sin conseguir su máximo de aprendizaje.

Ajustando los parámetros anteriormente mencionados, se puede conseguir que el algoritmo finalice en el momento que se sabe que es conveniente; alrededor del máximo donde la curva empieza a descender. En este caso, mirando la imagen 2 debe ser alrededor de 20-30 épocas.

Una vez ajustados los parámetros: **patience** = 5 y **min_delta** = 0.001 se ve la siguiente gráfica la cual se ajusta más a la realidad corrigiendo el sobre entrenamiento posible y no cortando antes de tiempo, para no perder capacidad de aprendizaje.

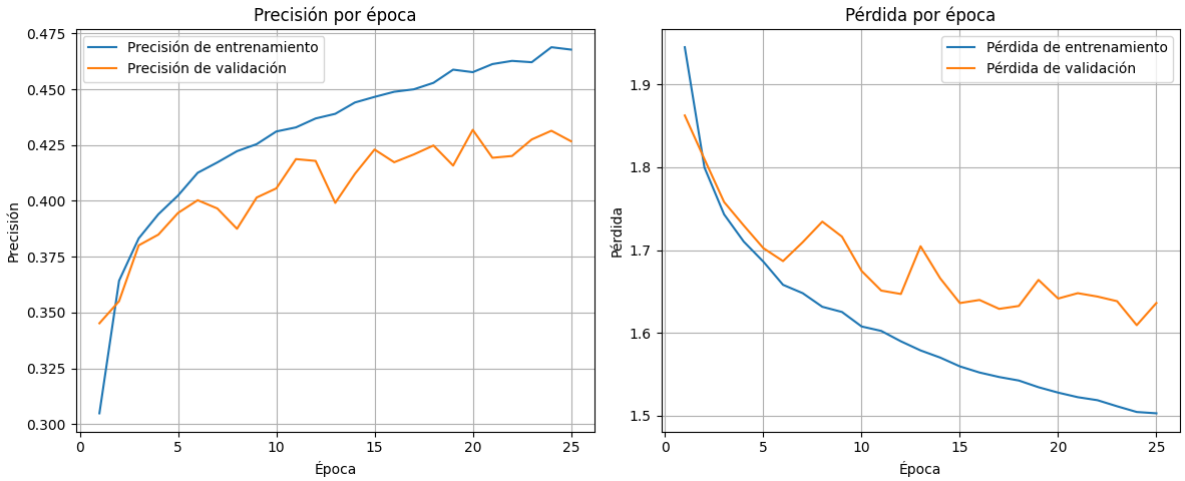


Ilustración 4(Early Stop ajustado)

Una vez ajustado **early stop** se pasa a ajustar el **split**.

El parámetro **split** hace referencia al porcentaje de datos que se usan para el entrenamiento y el que se usa para la validación. El porcentaje de entrenamiento debe ser mayor al de validación para que se consiga un buen resultado, normalmente se usa un 20% para validación y un 80% restante para entrenamiento.

Aquí se va a ir probando distintos porcentajes para ver cual nos da el mejor resultado.

Se hará con cada valor de **split,** 5 entrenamientos completos y después se calculará la media, esto es porque el algoritmo cada vez nos proporciona un resultado final distinto ya que es aleatorio. (Se ejecutará el entrenamiento haciendo uso de E*arly stop*)

En la siguiente gráfica *(Ilustración 5)* se pueden ver la comparación de cada tasa de acierto con el tamaño del *split*.

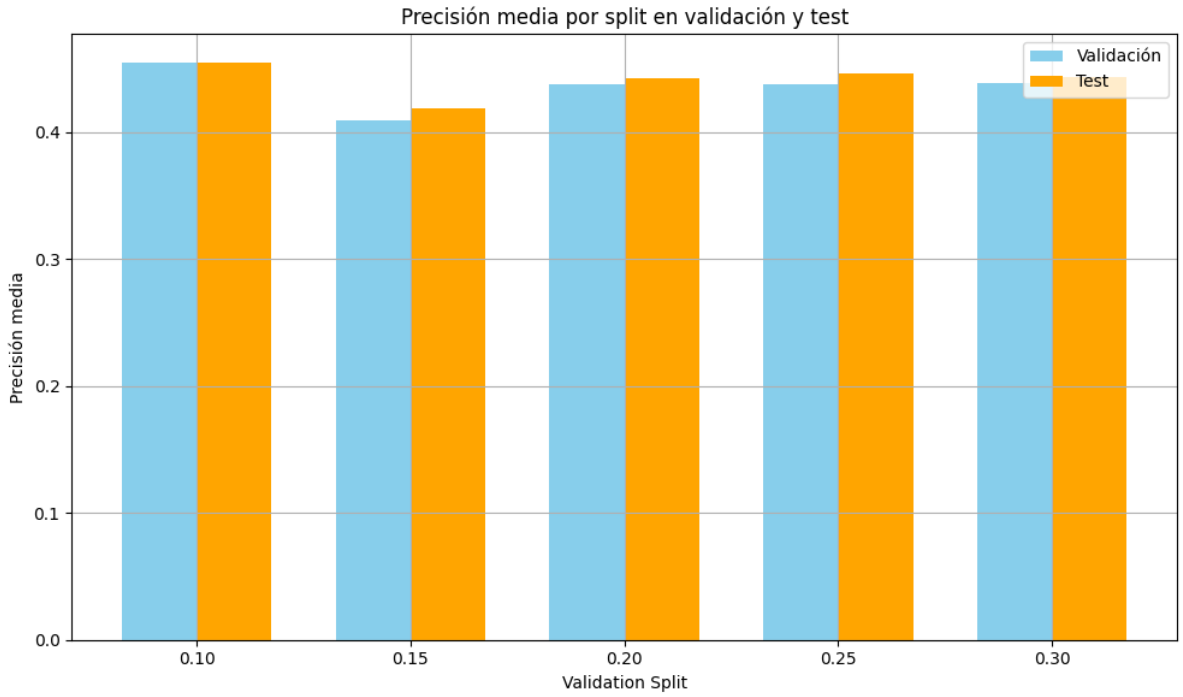


Ilustración 5 (Acierto en base a split)

Tarea C

Ahora ajustaremos el parámetro **batch_size,** el cual influye en diversas dinámicas: velocidad de entrenamiento, uso de memoria, generalización del modelo y tasa de aprendizaje.

Al aumentar el tamaño de este parámetro realmente estamos dividiendo las tareas de la red neuronal para aprovechar las capacidades de procesamiento paralelo del hardware que la entrene. Por ello, mejora significativamente el tiempo de entrenamiento. Ref4.

Por otro lado, con relación a **generalizar el modelo** un tamaño de *batch* menor puede introducir más ruido a la estimación; mientras que *batchs* más grandes proporcionan un ajuste más preciso del gradiente global.

Además con un mayor tamaño también se obtienen mejores resultados.

Pero no se puede aumentar su tamaño indefinidamente, a partir de cierto valor el entrenamiento deja de ser efectivo y se obtienen peores resultados aunque el tiempo de entrenamiento sea mucho menor.

Con el fin de obtener el valor óptimo se han generado las siguientes gráficas las cuales muestran entrenamientos con distintos valores de *batch_size*, su tasa de acierto y tiempo de ejecución:

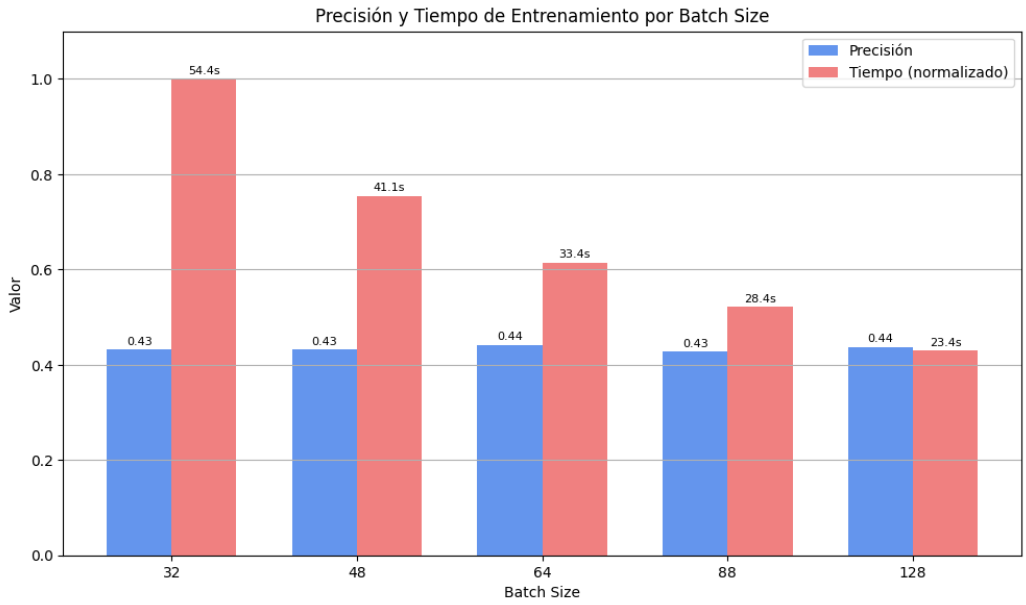


Ilustración 6

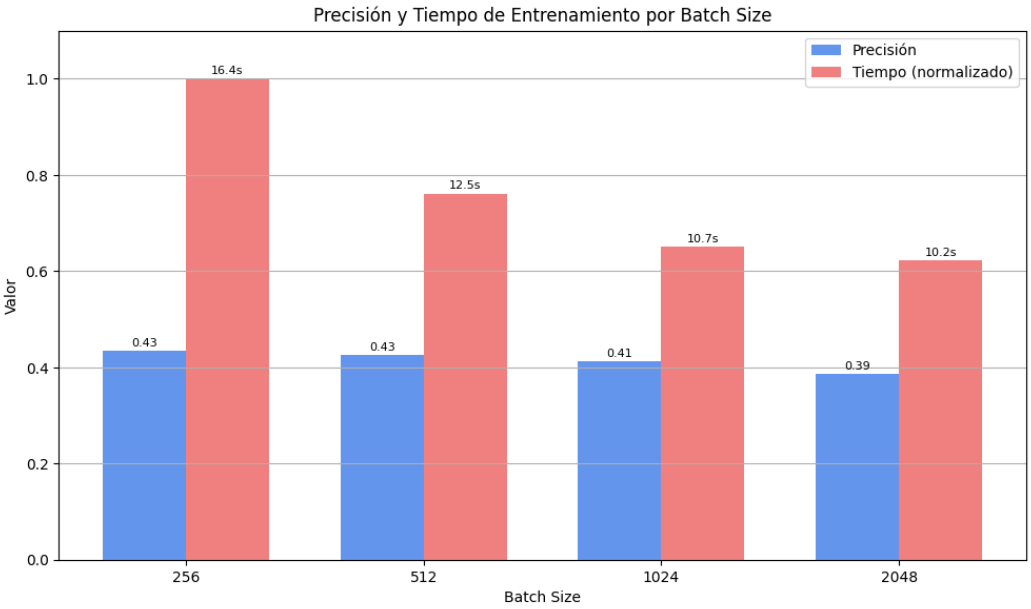


Ilustración 7

Como se ha mencionado anteriormente el tiempo de ejecución disminuye considerablemente manteniendo prácticamente constante la tasa de acierto.

Cuando se aumenta el batch por encima de los 256 comienza a disminuir la tasa de acierto.

También se observa que el mejor valor lo ofrece un valor de batch de 88, pero cabe remarcar que las pruebas se han hecho una vez por cada valor y al ser aleatorias no se debe tener en cuenta tal aumento porque podría ser puntual.

Por lo tanto, para una buena relación entre tiempo y tasa de acierto, el valor de *batch* debe establecerse entre los 128 y 265.

Para reforzar los resultados se han generado las matrices de confusión para valores de 32 y 256:

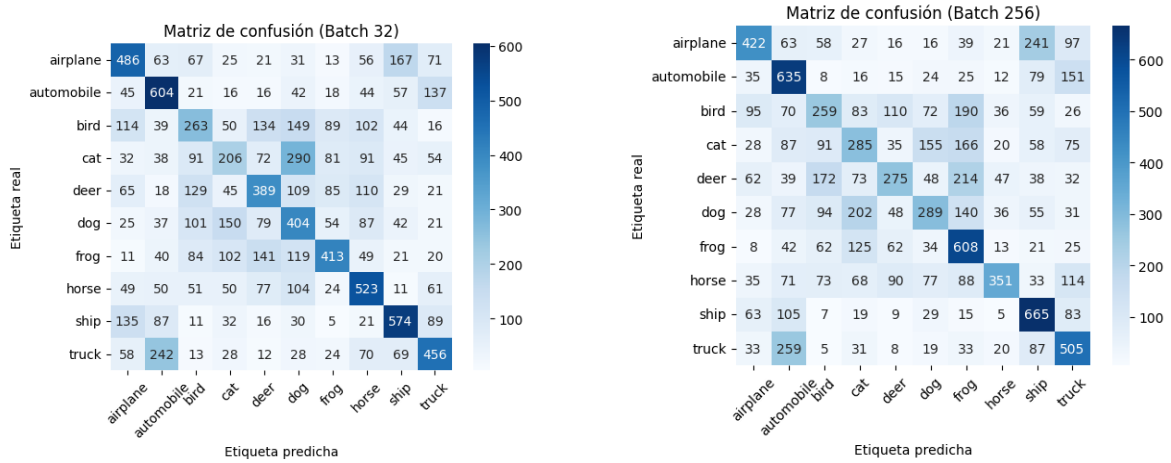


Ilustración 8(Tamaños 32 y 256 batch)

Se ve que los valores que se obtienen son realmente parecidos, dejando claro que hay una mejora de tiempo la cual no afecta a los resultados obtenidos.

Tarea D

Otro parámetro importante a la hora de definir la red MLP es la función de activación de las capas ocultas. Hay diversas funciones las cuales funcionarán mejor en unos casos que en otros.

Estas funciones se encargan de propagar la salida de los nodos hacia los siguientes, transforman la combinación de entradas, pesos y sesgos. Ref.

Las funciones con las que se probarán los entrenamientos son las siguientes:

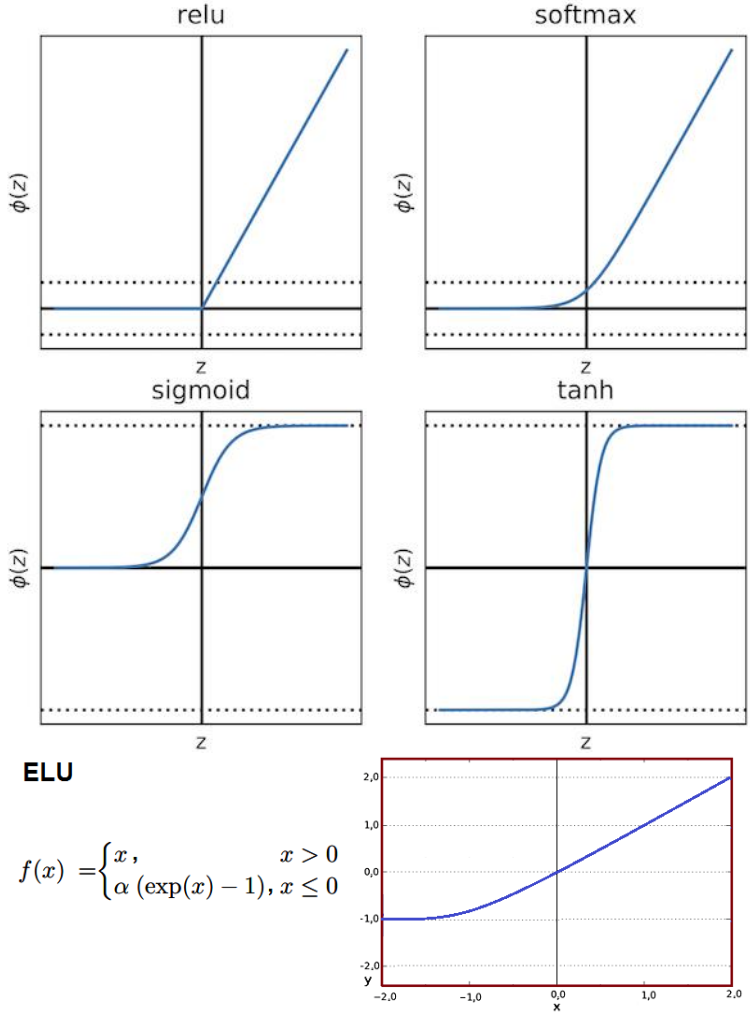


Ilustración 9(Funciones de Activación

Tras realizar entrenamientos con todas las funciones se han comparado los datos en un gráfico (*Ilustración 10*) el cual muestra el tiempo de ejecución y la tasa de acierto de cada una de las funciones de activación utilizadas.

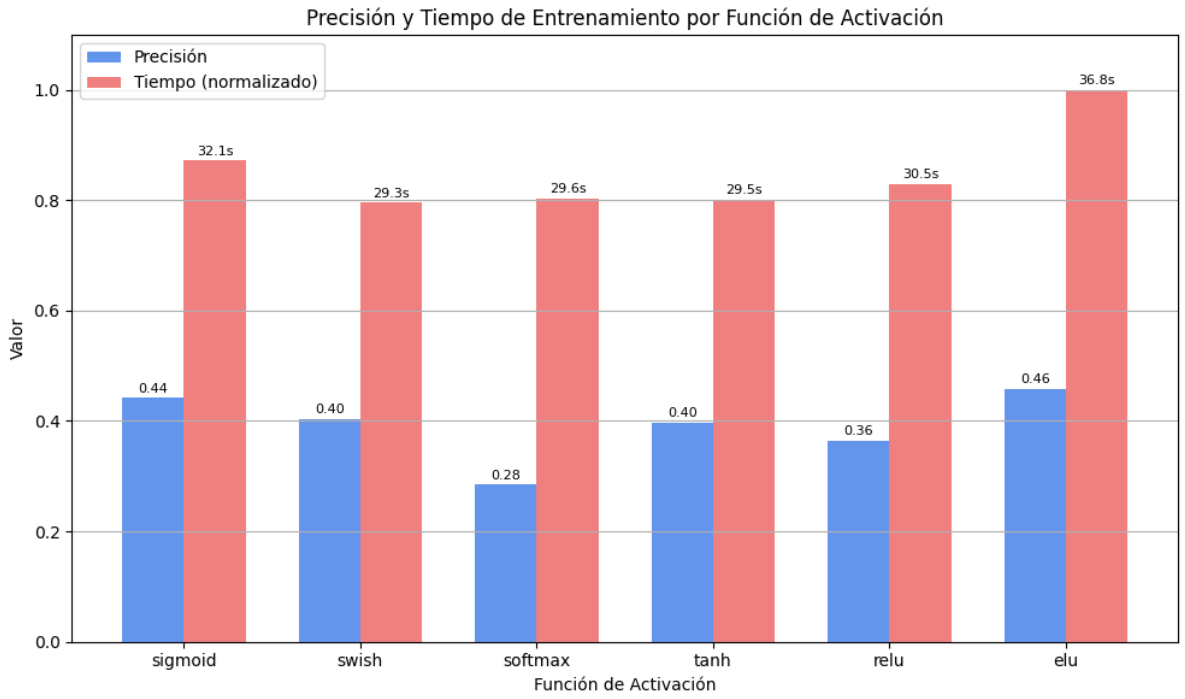


Ilustración 10 (Funciones de activación/tiempo/acierto)

Los datos muestran que la función "*elu*" es la que mejor tasa de acierto deja, en segundo lugar se encuentra "*sigmoid*". Pero según el criterio de tiempo de ejecución, *sigmoid* es más efectiva sacrificando décimas de resultado.

La que peor tasa entrega es "*softmax*" pero también la más rápida.

Aquí las matrices de confusión de las 3 funciones mencionadas:

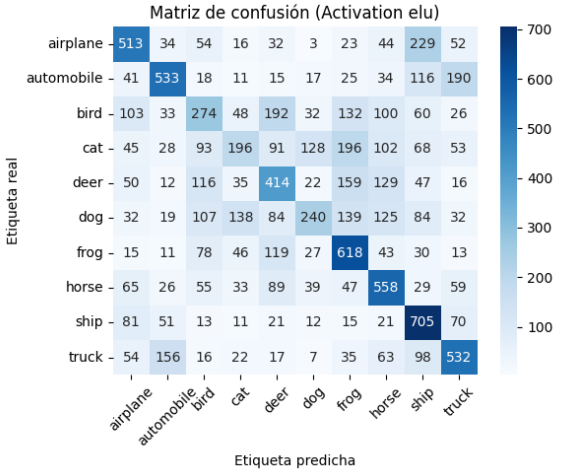


Ilustración 12 (Elu)

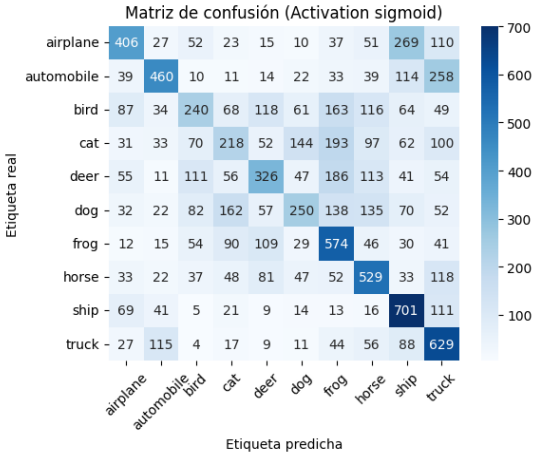


Ilustración 11 (Sigmoid)

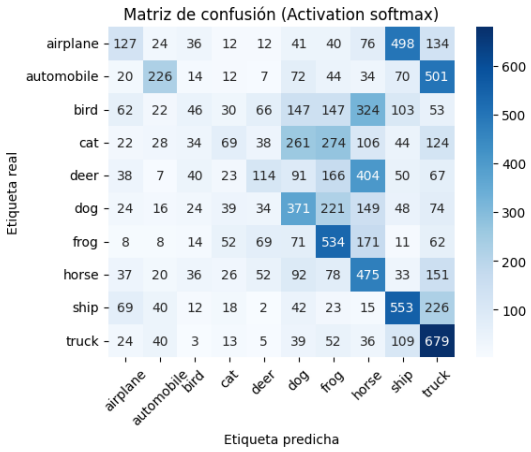


Ilustración 13 (Softmax)

Se puede ver que hay un error al analizar las clases: gato, pájaro y perro. Principalmente se suelen confundir todos los modelos más en estas 3 clases que en las demás. Un error que se debe probablemente a la naturaleza de las fotos y del objeto a identificar, fondos y el propio objeto más parecidos entre sí.

Tarea E

El número de neuronas en las capas ocultas influyen tanto en la tasa de acierto como en el tiempo de procesamiento. Por ello, en esta tarea se estudia cómo se puede mejorar la red neuronal mediante la variación de este parámetro.

Se han entrenado varios MLP variando únicamente el número de neuronas que forman una capa oculta de la red y se han obtenido los siguientes resultados:

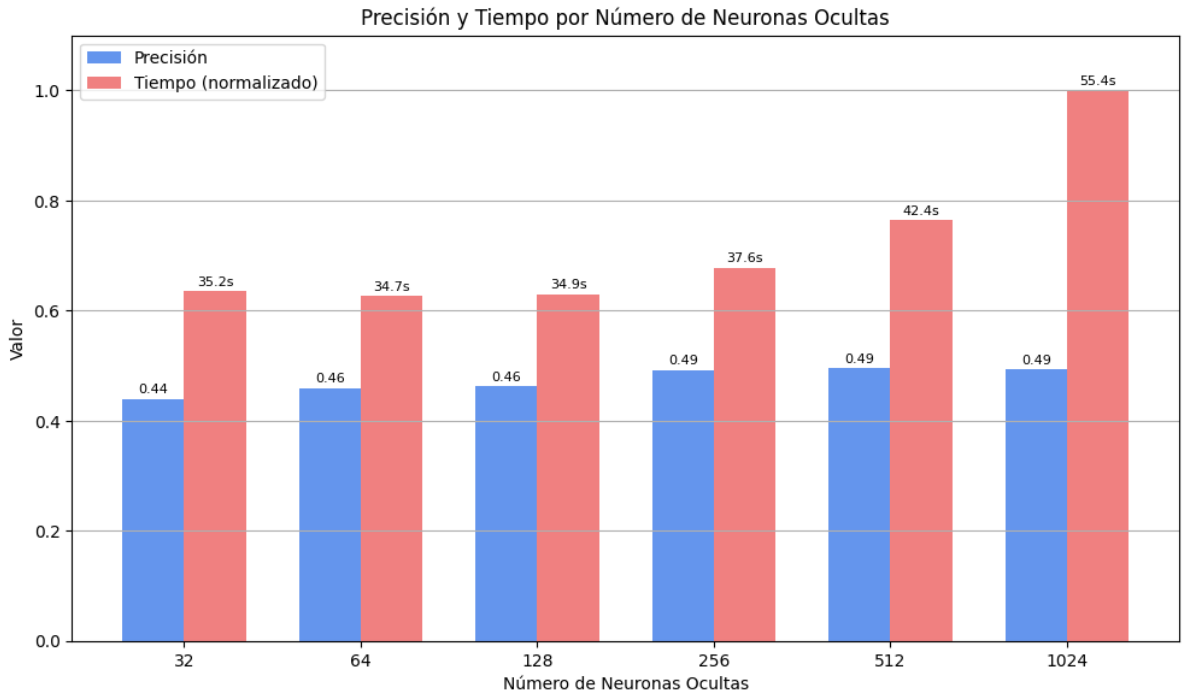


Ilustración 14

Como se puede apreciar (*ilustración 14*), hasta las 128 neuronas el tiempo de entrenamiento se mantiene constante; al llegar a 256 aparece una barrera en la tasa de acierto la cual no se consigue mejorar. Aunque se aumente el número de neuronas, los resultados no mejoran e incluye un aumento significativo del tiempo de procesado.

Por tanto, viendo la mejora en los resultados y que el tiempo de procesamiento es tan solo de 3 segundos usando 256 neuronas en vez de las 32 iniciales, se puede establecer ese valor para seguir mejorando el MLP.

Tarea F

En relación con la tarea anterior, ahora se añadirán capas ocultas para ver cómo influye en el tiempo de procesamiento y los resultados obtenidos.

Para ello, se han entrenado diferentes modelos. Se hacen comparaciones manteniendo el número total de neuronas pero variando la distribución en una o varias capas, además su ubicación.

1º Caso: 96 neuronas 2 capas

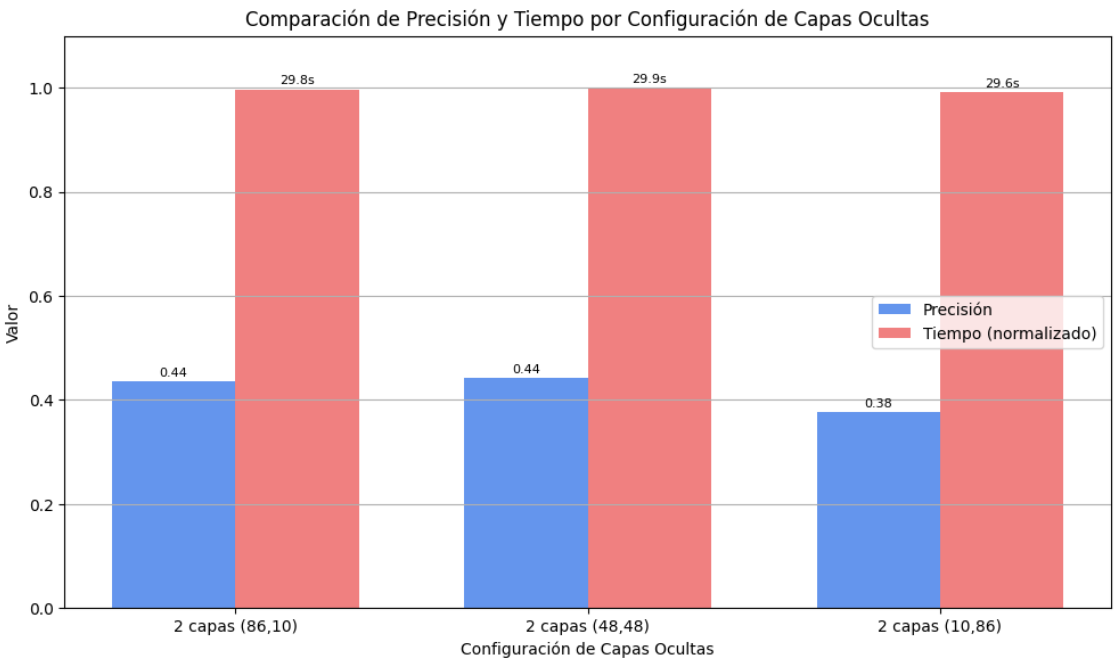


Ilustración 15(96 neuronas)

2º Caso: 96 neuronas 3 capas

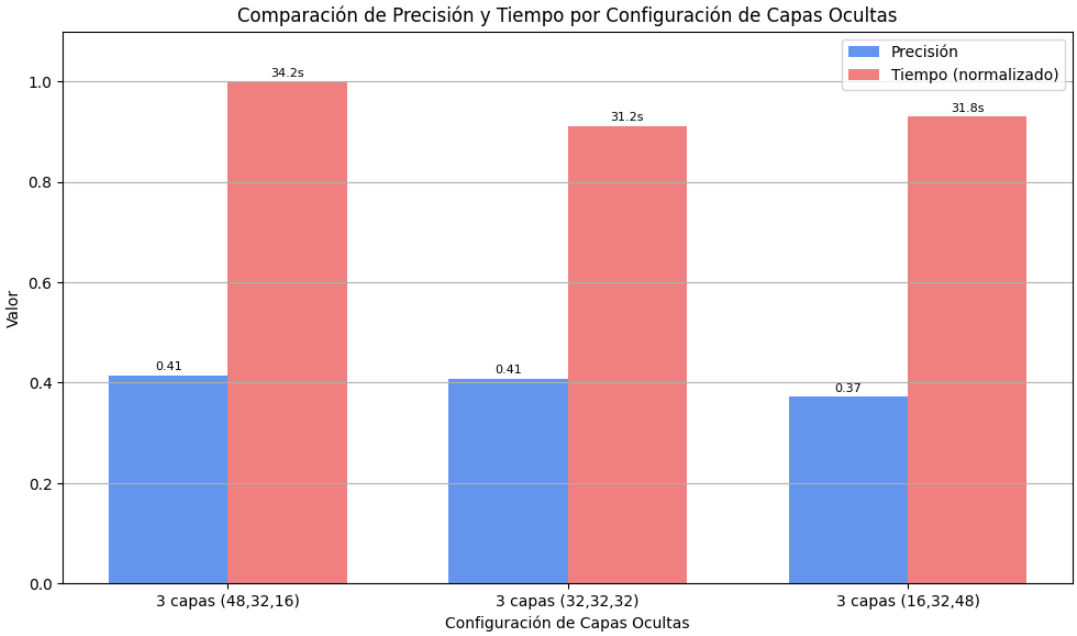


Ilustración 16 (96 neuronas

3º caso: 144 neuronas

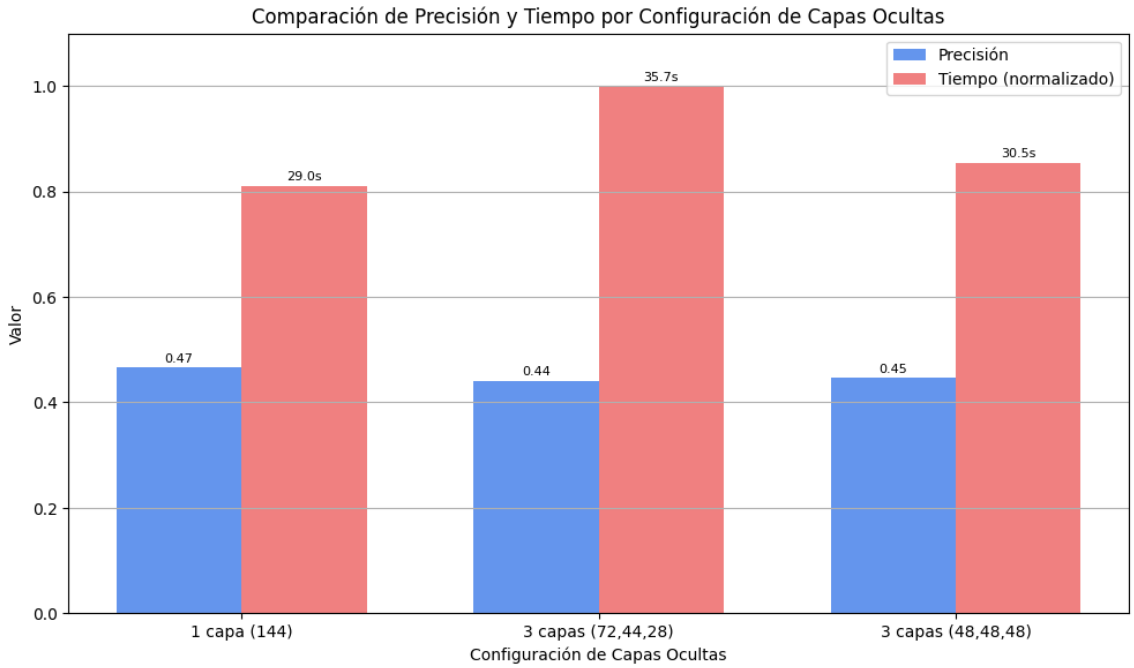


Ilustración 17 (144 neuronas)

4º caso: 256 neuronas

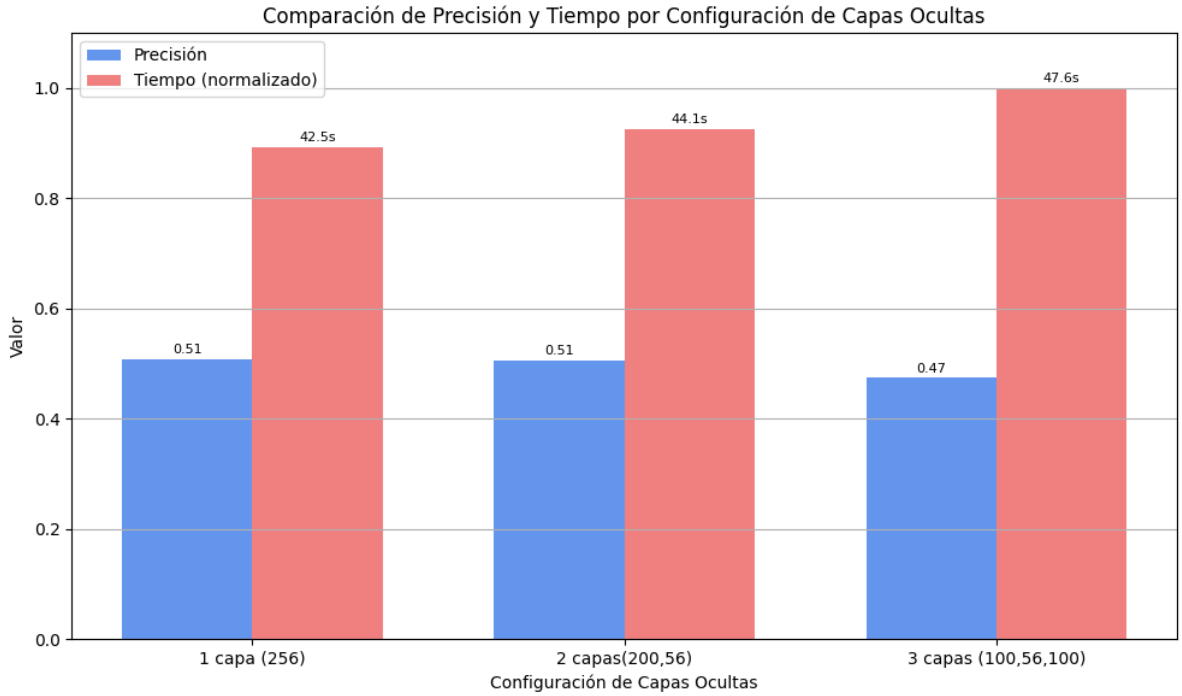


Ilustración 18 (256 neuronas)

En la tarea E se ha mencionado que los mejores resultados se obtienen con aproximadamente 256 neuronas, un valor el cual se puede confirmar al realizar los gráficos anteriores. (*ilustración 18*)

Por otro lado, la experimentación apunta a que aumentar el número de capas ocultas, en este caso no mejora los resultados obtenidos, incluso los empeora.

Para obtener los mejores resultados dividiendo en 2 capas ocultas, siempre son mejores cuando el número de neuronas de la primera capa es mayor que de la segunda, como muestra el caso 1 y el caso 2.

Redes Neuronales Convolucionales

A partir de ahora se va a trabajar con Redes Neuronales Convolucionales (CNN).

Las CNN son un tipo de red neuronal profunda y de avance la cual está especializada en resolver problemas con imágenes como clasificación, detección de objetos, segmentación, reconocimiento facial... Ref5.

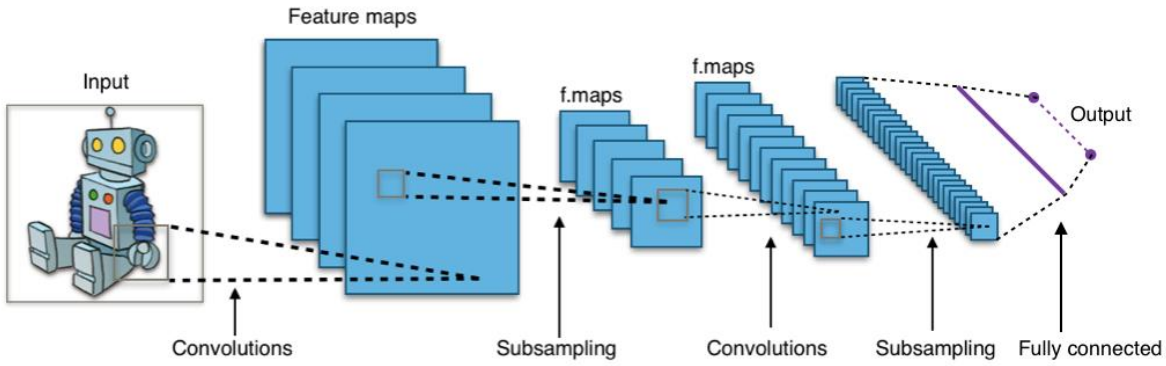


Ilustración 19 (Esquema CNN)

El uso de este tipo de red supone una mejora con respecto al MLP usado anteriormente. Pues se aplican filtros de convolución a las imágenes de entrada, permitiendo extraer características de más alto nivel teniendo en cuenta información espacial, adaptándose mejor a problemas de visión artificial.

Tarea G

Para crear la red convolucional se comienza declarando con dos capas Conv2D de 16 y 32 filtros respectivamente, con activación *relu* y filtros 3x3, y capa de salida Dense con función *softmax*. Además se implementa el uso de *Early Stopping* para no incurrir en sobre entrenamiento.

El modelo queda con la siguiente estructura:

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 16)	448
conv2d_1 (Conv2D)	(None, 28, 28, 32)	4,640
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 10)	250,890

Ilustración 20 (Estructura CNN primario)

El primer resultado que entrega esta red tras ser entrenada es el siguiente:

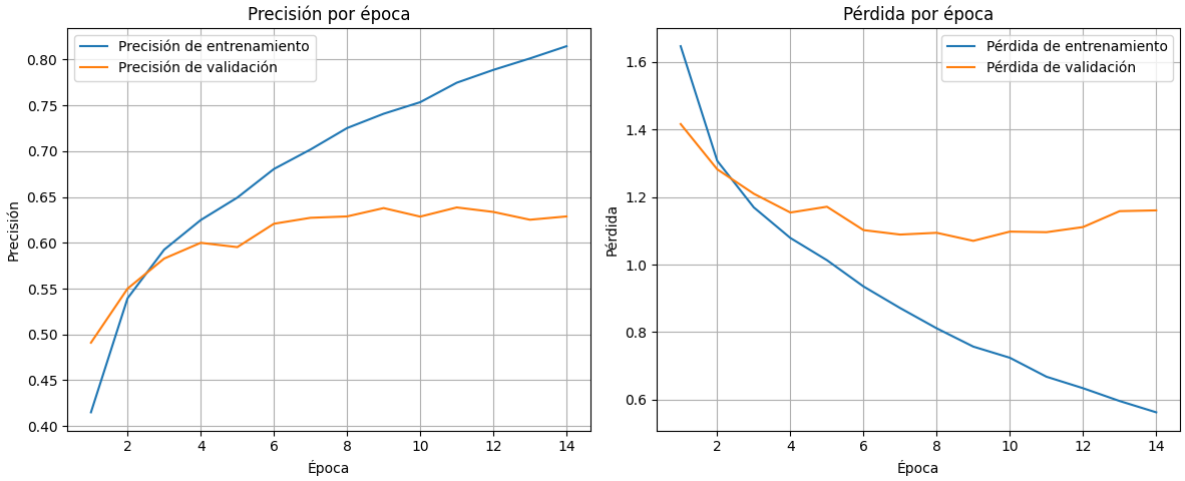


Ilustración 21 (Resultado CNN primario)

Se puede observar una mejoría con respecto al MLP de antes. Incluso antes de modificar los parámetros y ajustarlos.

Ahora se añade una capa de MaxPooling2D con tamaño de filtro 2x2 después de cada capa Conv2D.

El proceso de *MaxPooling* consiste en la discretización basado en muestras. Como objetivo, sub-muestrear una representación de entrada reduciendo su tamaño y su coste de cálculo.

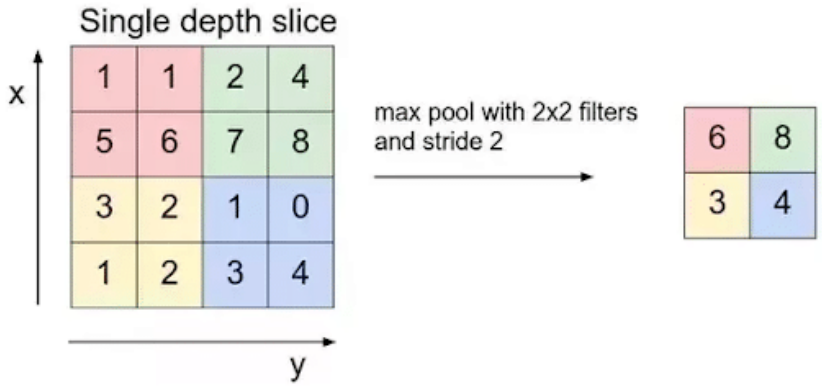


Ilustración 22 (Ejemplo MaxPooling)

El modelo que implementaría esta técnica quedaría definido de la siguiente forma:

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 30, 30, 16)	448
max_pooling2d (MaxPooling2D)	(None, 15, 15, 16)	0
conv2d_3 (Conv2D)	(None, 13, 13, 32)	4,640
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 32)	0
flatten_1 (Flatten)	(None, 1152)	0
dense_1 (Dense)	(None, 10)	11,530

Ilustración 23 (CNN con MaxPooling)

Y el resultado del entrenamiento:

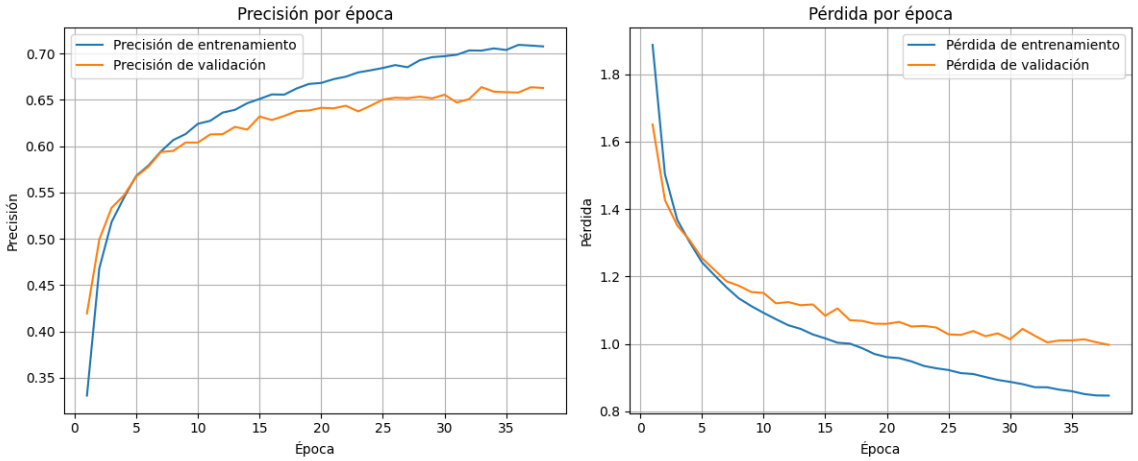


Ilustración 24 (Resultado CNN MP)

Observando estos resultados (ilustración 21 y 24) se ve claramente que la red sin *MaxPooling* tiene menos potencial de mejora, mientras que la que sí que utiliza estas capas se ve que puede entregar mejores resultados.

Tarea H

Para seguir mejorando el modelo y probando a modificar las características base, se compilarán distintos de ellos con diferente tamaño de filtro y salto, añadiendo o quitando capas *MaxPooling*.

(En todos los ejemplos se utiliza Early Stop)

La variación del **tamaño de filtro** (*kernel*) influye en que características puede detectar. Un filtro pequeño de 3x3 es capaz de detectar detalles locales más finos, mientras que un filtro más grande de 5x5 o 7x7 captará patrones más globales como estructuras y bordes grandes, además estos últimos son más costosos y presentan mayor riesgo de sobreajuste.

Por otro lado, el **stride** o **salto** afecta en cómo se va desplazando el filtro, si es pequeño (con valor 1) el salto será de 1 píxel por vez, lo que hace que sea más costoso pero más preciso. Si se aumenta este valor, captará menos datos y puede perder información menos perceptible, por el contrario es mucho más rápido.

En la ilustración 25 se muestra la comparativa al usar los diferentes tamaños de filtro con un salto unitario.

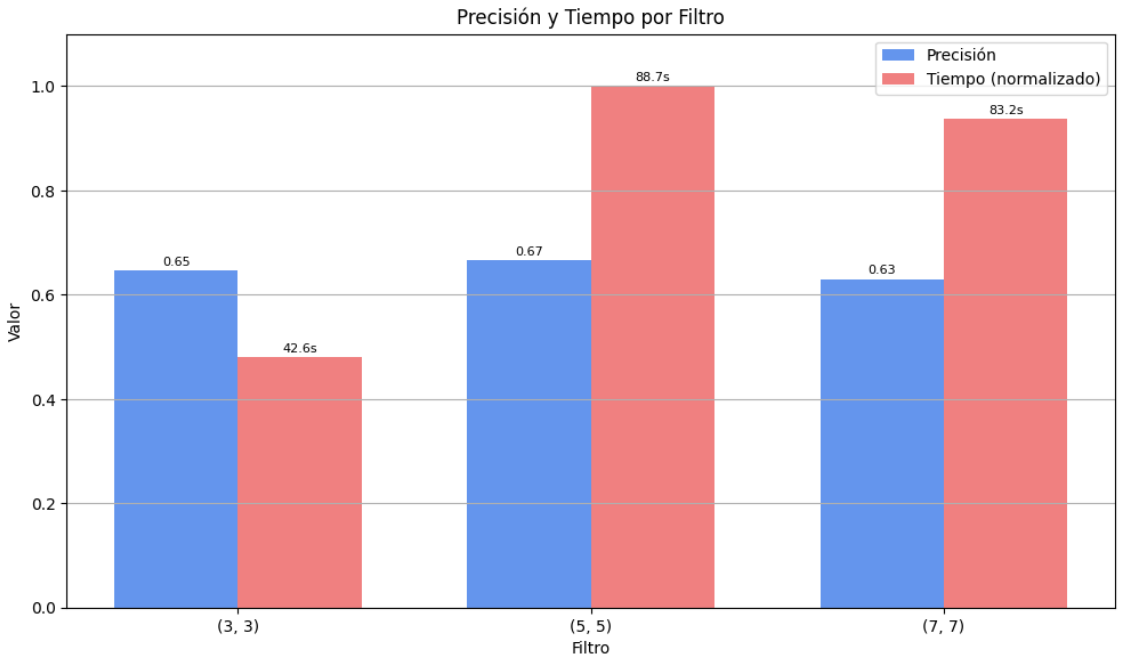


Ilustración 25 (diferentes filtros)

La tasa de acierto aunque es superior al usar un filtro de 5x5 que de 3x3, el tiempo de procesamiento es casi el doble. Por tanto, lo mejor sería hacer uso del filtro más pequeño.

Y ahora en la ilustración 26 los mismos tamaños de filtro pero con un salto 2.

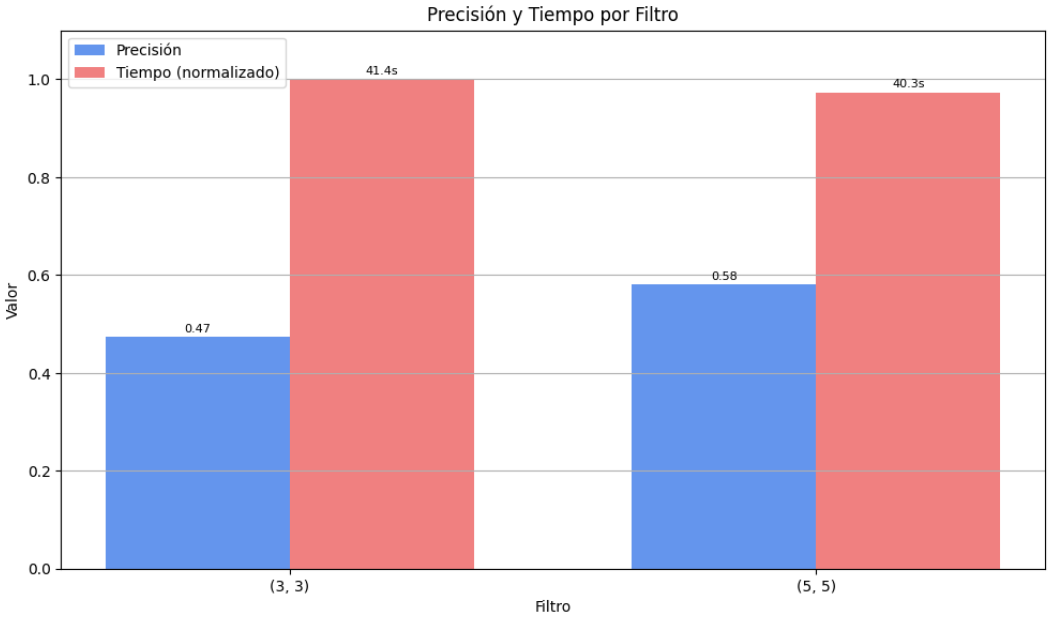


Ilustración 26 (filtros stride 2)

La diferencia con respecto al primer ejemplo es notoria, los tiempos de procesamiento son similares pero la tasa de acierto es superior con filtro de 5x5, sin embargo al usarse un salto más grande, la capacidad de detección de detalles disminuye y se ve reflejado en la calidad del entrenamiento (ambos resultados son menores que antes).

Por otro lado, se puede explicar que funcione mejor un filtro mejor que otro mediante la matriz de confusión:

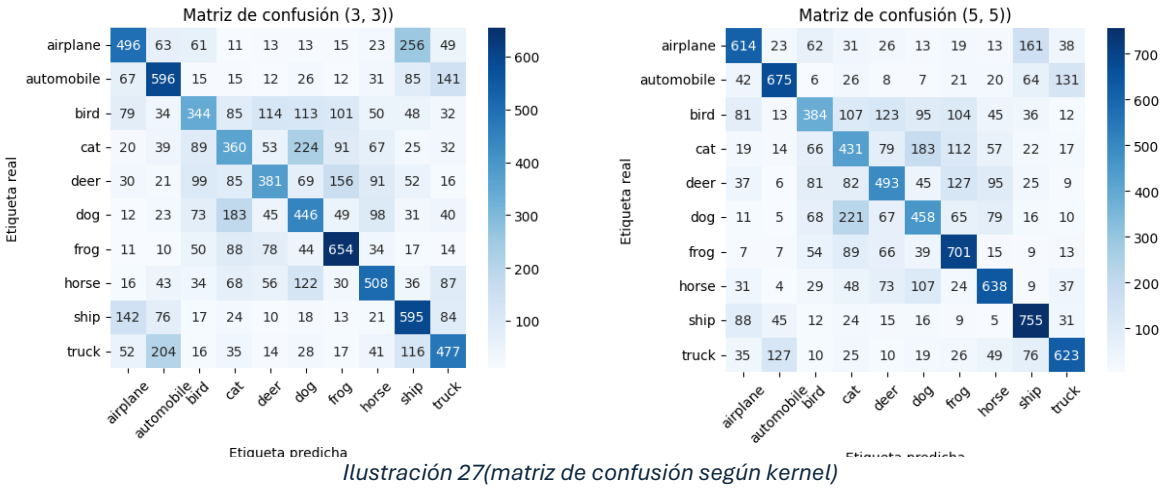


Ilustración 27(matriz de confusión según kernel)

Hay casos en los que tener características globales viene mejor que tener algunas más concretas, son elementos más fácilmente separables por formas grandes. Por eso la tasa de acierto global es mejor.

Tarea Opcional

En este punto se quiere obtener una red convolucional que optimice la arquitectura del modelo. Durante toda la práctica se han estado variando diversos parámetros de las redes neuronales con el fin de mejorar algunos aspectos, ahora se realizarán las pruebas y cambios necesarios para conseguir los mejores resultados. Ref6

Para empezar, se modificarán los filtros que entran, para conseguir un resultado más preciso se usara 3x3. La función de activación será '*relu*' y el número de filtros subirá a 32 cada capa. El modelo se vería así: Ej1

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 32)	9,248
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 32)	0
flatten (Flatten)	(None, 2048)	0
dense (Dense)	(None, 10)	20,490

Ilustración 28 (Modelo ej1)

Resutado: accuracy: 0.6798 - loss: 0.9364 Accuracy: 0.6779 - Tiempo: 51.17 s

Para mejorar el resultado se añade una normalización del *batch* tras las capas Conv2D y se aumenta el número de filtros. Este último valor hace referencia al número de características que puede detectar. Ej3

Resultado: accuracy: 0.6796 - loss: 1.9862 Accuracy: 0.6790 - Tiempo: 404.21 s

Al aumentar el número de filtros, los datos que tiene que procesar aumentan significativamente y eso se puede ver en el tiempo de procesado. Además no necesariamente mejoran la efectividad del entrenamiento.

Con 4 capas Conv2D, aplicando normalización de lote y MaxPooling, se aumenta la tasa de acierto pero también hay mayor tiempo. Ej2

Resultado: accuracy: 0.7208 - loss: 0.9435 Accuracy: 0.7189 - Tiempo: 102.13 s

Link al código de los ejemplos mencionados. *ref*

El mejor resultado se ha conseguido en el ejemplo 2 con un accuracy de 72% y una pérdida de 0.94.

Se puede ver una demostración de funcionamiento del modelo final entrenado, con imágenes y su predicción.



Conclusiones

El desarrollo de redes neuronales y/o modelos de aprendizaje supervisado puede ser clave para la aparición de tecnologías y sistemas inteligentes. Como se ha podido demostrar durante toda la experimentación, un buen ajuste de todos los componentes de la red es determinante para que tenga la mejor probabilidad de acierto.

Se ha podido comparar los dos tipos de redes, la convolucional y la basada en el perceptrón multicapa.

En el contexto de la práctica, para la clasificación de imágenes el MLP básico no ofrece muy buenos resultados, (como mucho un 50% de aciertos) y tampoco se puede esperar una gran mejora.

Por otro lado, los modelos de redes convolucionales tienen más potencial para poder hacer una buena clasificación, se ha visto el nivel de mejora de la red, alcanzando un 72% de acierto. Lo cierto es que se podría seguir mejorando, pero haría falta un hardware más potente para reducir los tiempos de procesado y mejorar la calidad del modelo.

Por consiguiente, queda demostrada la eficacia de generar un buen modelo que se adapte a las características del problema a resolver. Siendo el MLP un modelo más básico y que puede funcionar mejor para otro tipo de tareas y el modelo convolucional muy bueno para la clasificación de imágenes, puesto que se basa en la convolución de estas.

Bibliografía:

Cifar 10: https://docs.ultralytics.com/es/datasets/classify/cifar10/ (Ref1)

Redes neuronales: https://www.unir.net/revista/ingenieria/redes-neuronales-artificiales/ (Ref2)

Funcionamiento del MLP: https://www.datacamp.com/es/tutorial/multilayer-perceptrons-in-machine-learning (Ref3)

Batch size: https://www.ultralytics.com/es/glossary/batch-size (Ref4)

CNN: https://www.tensorflow.org/tutorials/images/cnn?hl=es-419 (Ref5)

CNN: https://www.datacamp.com/es/tutorial/convolutional-neural-networks-python (Ref5)

Ejemplos: https://gist.github.com/AitorRuizB/4e785bb967dcacd16a7de7013c9f342d

Implementación CNN: https://es.linkedin.com/pulse/implementaci%C3%B3n-de-una-cnn-resnet-34-usando-keras-gu%C3%ADa-jose-luis-2rwff

Funciones Activación: https://telefonicatech.com/blog/las-matematicas-del-machine-learning-funciones-de-activacion#:~:text=Las%20funciones%20de%20activaci%C3%B3n%20se,no%20lineales%20a%20la%20red.